

Physics Note: Pion Energy-Momentum Tensor Measurement

PyQUDA Meson EMT Workflow – Connected 3pt, Quark 1pt, and Gluon 1pt

PyQUDA Collaboration

May 5, 2026

Contents

1	Introduction and Physics Motivation	3
2	Code Architecture Overview	3
3	Part I: Connected Meson EMT	4
3.1	Meson Two-Point Function	4
3.2	Connected Three-Point Function (before sequential trick)	5
3.3	Scalar Diagnostic: C_3^χ	5
3.4	Connected Quark EMT Insertion	5
3.4.1	First term: derivative on the forward (source-to-insertion) line	6
3.4.2	Second term: derivative on the backward (sequential) line	6
3.4.3	Final contraction and symmetrization	6
3.5	Fixed-Sink Sequential Source Construction	7
3.6	Symmetric Covariant Derivative on a Propagator	8
3.7	Left Derivative on the Sequential Line	8
3.8	Gradient Flow	9
3.8.1	Flow schedule	9
3.8.2	Flowing propagators together	9
3.8.3	Gauge field flow	9
3.9	Connected 3pt High-Level Algorithm	9
3.10	HDF5 Output Format (Connected 3pt)	10
4	Part II: Disconnected One-Point Functions	10
4.1	Stochastic Quark One-Point Function	10
4.1.1	Noise generation	10
4.1.2	CHI diagnostics	10
4.1.3	Quark EMT one-point building block	11
4.1.4	Symmetrization, noise averaging, and volume normalization	11
4.1.5	HDF5 output format (Quark 1pt)	11
4.2	Ringed Fermion Normalization	11
4.3	Gluon One-Point Function	12
4.3.1	Clover field strength	12
4.3.2	Traceless projection	12
4.3.3	Gluon EMT building block	13
4.3.4	HDF5 output format (Gluon 1pt)	13
4.4	Gradient Flow Schedule (One-Point)	13
4.5	Renormalization Assembly (Analysis Stage)	14

4.6	Future Variance Reduction Strategies	14
4.7	Summary of All HDF5 Output Files	14
5	Technical Implementation Details	15
5.1	Parameter Dictionary	15
5.2	Inverter Parameters	15
5.3	Random Number Parameters (Quark 1pt)	15
5.4	Typical Usage Pattern	15
6	Cross-Checks and Consistency	16

1 Introduction and Physics Motivation

The energy-momentum tensor (EMT) $T_{\mu\nu}(x)$ is a fundamental operator in quantum field theory. In QCD it encodes the distribution of energy, momentum, pressure, and shear stress inside hadrons. Its matrix elements between hadron states,

$$\langle \pi(p_f) | T_{\mu\nu} | \pi(p_i) \rangle, \quad (1)$$

determine the gravitational form factors $A(q^2)$, $B(q^2)$, $D(q^2)$, and $\bar{c}(q^2)$ of the pion. These form factors provide access to the mass and spin decomposition of the pion, its mechanical radius, and the D -term (pressure distribution). Unlike the proton, the pion has spin zero, simplifying the Lorentz decomposition.

On the lattice, $T_{\mu\nu}$ requires a non-perturbative renormalization scheme. The gradient flow [1, 2] provides a regularization-independent method: flowing gauge and fermion fields to finite flow time t renders the bare composite operators finite. At small flow time, flowed operators are matched to $\overline{\text{MS}}$ via perturbatively computed matching coefficients $c_q(t)$ and $c_g(t)$:

$$T_{\mu\nu}^{\text{ren}}(x) = c_g(t) O_{\mu\nu}^g(t, x) + c_q(t) O_{\mu\nu}^q(t, x) + \text{mixing/trace terms}. \quad (2)$$

The mixing must account for the fact that the trace of $T_{\mu\nu}$ receives contributions from the trace anomaly. The matching coefficients $c_q(t)$, $c_g(t)$ are computed in perturbation theory and are part of the analysis stage, not the contraction kernel.

This note documents the complete numerical implementation in PyQUDA of:

- Connected meson three-point functions with quark EMT insertion (Section 3),
- Stochastic quark one-point functions for disconnected diagrams (Section 4.1),
- Gluon one-point functions (Section 4.3),

together with the gradient-flow schedule and HDF5 output formats. The target audience is lattice practitioners who need a line-by-line correspondence between the continuum formulas and the einsum contraction patterns in the Python source.

Key conventions

Throughout this note:

- Euclidean metric with γ_4 hermitian: $\gamma_\mu^\dagger = \gamma_\mu$, $\{\gamma_\mu, \gamma_\nu\} = 2\delta_{\mu\nu}$.
- $\gamma_5 = \gamma_1\gamma_2\gamma_3\gamma_4 = \gamma_5^\dagger$.
- Gamma hermiticity of the quark propagator: $S(x, y)^\dagger = \gamma_5 S(y, x) \gamma_5$.
- Convention B meson contraction with `meson_sign` = +1.
- Source position $x_0 = (t_0, \vec{x}_0)$, insertion $x = (\tau, \vec{x})$, sink $y = (t_{\text{sep}}, \vec{y})$.
- Momentum phase: $\Phi_k(r - x_0) = e^{ik \cdot (r - x_0)}$ where k uses continuum normalization $k_\mu \in \frac{2\pi}{L_\mu} \mathbb{Z}$.

2 Code Architecture Overview

The class hierarchy is:

<code>EMTDisconnectedQuark1pt</code>	shared quark-1pt loop
<code>EMTDisconnectedGluon1pt</code>	shared gluon-1pt loop
<code>QuarkEMT(EMTDisconnectedQuark1pt)</code>	connected meson 3pt + 2pt
<code>GluonEMT(EMTDisconnectedGluon1pt)</code>	alias for gluon 1pt

`QuarkEMT` inherits all stochastic 1pt methods (including covariant derivatives, gamma5 hermiticity, and flow utilities) while adding meson-specific two-point and three-point contractions.

The application-level entry points are:

- `application/EMT_meson/perlmutter/Pyquda.EMT_quark.3pt.py` – connected 3pt
- `application/EMT_meson/perlmutter/Pyquda.EMT_quark.1pt.py` – quark 1pt

- `application/EMT_meson/perlmutter/Pyquda.EMT_gluon.1pt.py` – gluon 1pt

3 Part I: Connected Meson EMT

3.1 Meson Two-Point Function

The physical connected meson two-point function with source interpolator Γ_{src} and sink interpolator Γ_{sink} is

$$C_2^{\Gamma_{\text{sink}}}(p, t) = \sum_{\vec{x}} \Phi_{-p}(x - x_0) \text{Tr}_{sc} \left[S_{\text{anti}}(x, x_0) \Gamma_{\text{sink}} S_q(x, x_0) \Gamma_{\text{src}} \right], \quad (3)$$

where the antiquark line is built using gamma5 hermiticity:

$$S_{\text{anti}}(x, x_0) = \gamma_5 S_q(x, x_0)^\dagger \gamma_5. \quad (4)$$

In the code (`contract_meson_2pt`), the independent backward propagator `prop_bw` is used to form the antiquark line:

$$\text{bw_prop}(x) = \gamma_5 \text{prop_bw}(x, x_0)^\dagger \gamma_5, \quad (5)$$

and the contraction is computed as

$$C_2 = \sum_{\vec{x}} \Phi_{-p}(x - x_0) \text{Tr}_{sc} [\text{bw_prop}(x) \Gamma_{\text{sink}} \text{prop_fw}(x, x_0) \Gamma_{\text{src}}]. \quad (6)$$

The implementation scans all 16 gamma structures for the sink while keeping Γ_{src} fixed. The 16 gamma matrices (in `my_gammas`) are:

$$\begin{aligned} \text{"5"} &= \gamma_5, & \text{"T"} &= \gamma_4, \\ \text{"T5"} &= \gamma_4 \gamma_5, & \text{"X"} &= \gamma_1, \\ \text{"X5"} &= \gamma_1 \gamma_5, & \text{"Y"} &= \gamma_2, \\ \text{"Y5"} &= \gamma_2 \gamma_5, & \text{"Z"} &= \gamma_3, \\ \text{"Z5"} &= \gamma_3 \gamma_5, & \text{"I"} &= \mathbf{1}, \\ \text{"SXT"} &= \sigma_{14}, & \text{"SXY"} &= \sigma_{12}, \\ \text{"SXZ"} &= \sigma_{13}, & \text{"SYT"} &= \sigma_{24}, \\ \text{"SYZ"} &= \sigma_{23}, & \text{"SZT"} &= \sigma_{34}, \end{aligned}$$

with $\sigma_{\mu\nu} = \frac{i}{2}[\gamma_\mu, \gamma_\nu]$. The ordering is fixed by `pyquda_gammas_order`: [15, 8, 7, 1, 14, 2, 13, 4, 11, 0, 9, 3, 5, 10, 6, 12].

In the einsum contraction, the data layout is

$$[\text{parity}, \text{t}, \text{z}, \text{y}, \text{x}, \text{spin_src}, \text{color_src}, \text{spin_sink}, \text{color_sink}]$$

so the einsum pattern for forming `bw_prop` is

$$\text{bw_prop} = \text{contract}(\text{"ij, wtzyxilab, kl} \rightarrow \text{wtzyxkjba"}, \text{G5, prop_bw.conj(), G5})$$

where index `i,j,k,l` $\in \{0, 1, 2, 3\}$ (spin) and `a,b` $\in \{0, 1, 2\}$ (color). The transpose of spin-sink and spin-source indices (`ilab` $\rightarrow$ `kjba`) implements the dagger with spin-color reordering.

The final scalar folding into C_2 reads

$$\begin{aligned} \text{bw_prop} &= \text{contract}(\text{"wtzyxjicf, gim} \rightarrow \text{gwtzyxjmcf"}, \text{bw_prop, sink_gammas}) \\ \text{scalar} &= \text{contract}(\text{"gwtzyxjiab, wtzyxilba, lj} \rightarrow \text{gwtzyx"}, \\ &\quad \text{bw_prop, prop_fw, src_gamma}) \end{aligned}$$

where the first line applies all 16 sink gamma matrices via a stacked index **g**, and the second line traces over spin-sink (**i**), spin-source (**1,j**), and color (Tr_c via repeated **a,b**).

Boosted smearing: If **CG_GaussSmear** is enabled, Gaussian smearing with a momentum boost is applied at both source and sink:

- Source smearing uses boost **pos_boost** on the forward source and **neg_boost** on the backward source (when they differ).
- Sink smearing is applied during the 2pt contraction with the same **pos_boost** and **neg_boost**.

3.2 Connected Three-Point Function (before sequential trick)

For a local quark bilinear insertion $\mathcal{O}_g(x)$ at the space-time point $x = (\tau, \vec{x})$ and a final sink with momentum p_f , the connected three-point function is

$$C_3^g(q, \tau; p_f, t_{\text{sep}}) = \sum_{\vec{x}} \sum_{\vec{y}} \Phi_q(x-x_0) \Phi_{p_f}(y-x_0) \text{Tr}_{sc} \left[S_{\text{anti}}(y, x_0) \Gamma_{\text{sink}} S_q(y, x) \mathcal{O}_g(x) S_q(x, x_0) \Gamma_{\text{src}} \right]. \quad (7)$$

Here q is the momentum transfer (denoted **qext** in the code), p_f is the fixed sink momentum (**pf**), and t_{sep} is the source-sink separation. The notation follows the usual three-point pattern: the quark propagates from source x_0 to insertion x , the operator is inserted, the quark continues to sink y , and the antiquark runs directly from source to sink.

3.3 Scalar Diagnostic: C_3^χ

With $\mathcal{O}_g(x) = \mathbf{1}$ (the unit operator in spin-color space), the three-point function reduces to

$$C_3^\chi(q, \tau) = \sum_{\vec{x}} \Phi_q(x-x_0) \text{Tr}_{sc} \left[S_{\text{seq_anti}}(x) S_q(x, x_0) \Gamma_{\text{src}} \right], \quad (8)$$

where $S_{\text{seq_anti}}(x)$ is the backward sequential meson line defined in Section 3.5. This diagnostic verifies that the sequential source construction is correct: a well-constructed sequential source should give C_3^χ that is numerically consistent with the two-point function when the scalar insertion is trivial.

In **get_C3_chi**, the contraction is

```
dst2 = make_dst2(seq_bw_prop) # gamma5 * S_seq^dag * gamma5
scalar_field = contract("wtzyxabij, wtzyxbcji, ca -> wtzyx",
                        dst2, prop_fw.data, src_gamma)
# Fourier transform to momentum space, keep time
slice_t = gatherLattice( contract("qwtzyx, wtzyx -> qt",
                                phases_3pt, scalar_field) )
```

Note the spin indices: **dst2** has layout (... , **a=spin_sink**, **b=spin_src**, **i=color_sink**, **j=color_src**), while **prop_fw.data** has (... , **b=spin_src**, **c=spin_sink**, **j=color_src**, **i=color_sink**). The contraction **abij**, **bcji**, **ca** traces over spin and color, with **src_gamma** providing the source interpolator.

3.4 Connected Quark EMT Insertion

The Euclidean connected quark EMT insertion is

$$T_{\mu\nu}^q(x) = \frac{1}{2} \left[\gamma_\nu D_\mu - \overleftarrow{D}_\mu \gamma_\nu \right], \quad (9)$$

where D_μ is the symmetric covariant derivative acting to the right and $\overleftarrow{D}_\mu$ acts to the left. This bilinear is not Hermitian by itself; the symmetrization $\mu \leftrightarrow \nu$ is performed after contraction to obtain a Hermitian EMT.

When inserted into the three-point function, two distinct contributions arise depending on which quark line the derivative acts upon:

3.4.1 First term: derivative on the forward (source-to-insertion) line

$$C_3^{\mu\nu, q, \text{first}}(q, \tau) = +\frac{1}{2} \sum_{\vec{x}} \Phi_q(x - x_0) \text{Tr}_{sc} \left[S_{\text{seq-anti}}(x) \gamma_\nu (D_\mu S_q(x, x_0)) \Gamma_{\text{src}} \right]. \quad (10)$$

The derivative acts on the forward propagator *before* multiplication by the gamma matrix. The overall sign is $+1/2$.

3.4.2 Second term: derivative on the backward (sequential) line

$$C_3^{\mu\nu, q, \text{second}}(q, \tau) = -\frac{1}{2} \sum_{\vec{x}} \Phi_q(x - x_0) \text{Tr}_{sc} \left[(\overleftarrow{D}_\mu S_{\text{seq-anti}}(x)) \gamma_\nu S_q(x, x_0) \Gamma_{\text{src}} \right]. \quad (11)$$

The derivative acts on the sequential backward propagator *from the left*. The overall sign is $-1/2$, which follows from the minus sign in Eq. (9) for the left-derivative term.

3.4.3 Final contraction and symmetrization

The full quark EMT three-point function is the sum of the two contributions:

$$C_3^{\mu\nu, q}(q, \tau) = C_3^{\mu\nu, q, \text{first}}(q, \tau) + C_3^{\mu\nu, q, \text{second}}(q, \tau). \quad (12)$$

After this sum, explicit symmetrization enforces $T_{\mu\nu} = T_{\nu\mu}$:

$$C_3^{\mu\nu, q}(q, \tau) \rightarrow \frac{1}{2} (C_3^{\mu\nu, q}(q, \tau) + C_3^{\nu\mu, q}(q, \tau)). \quad (13)$$

The code in `get_C3_Tmunu_symmetrized` implements this as:

```
# First term: +1/2 * Tr[dst2 * gamma_nu * (D_mu prop_fw) * Gamma_src]
for mu in range(4):
    D_fw = covdev_sym_prop(U_f, prop_fw, mu)
    for nu in range(4):
        gamma_D_fw = contract("ab, wtzyxbdi j -> wtzyxadi j",
                               D_gammas[nu], D_fw.data)
        scalar = 0.5 * contract("wtzyxabij, wtzyxbcji, ca -> wtzyx",
                                dst2, gamma_D_fw, src_gamma)

# Second term: -1/2 * Tr[(leftD_mu dst2) * gamma_nu * prop_fw * Gamma_src]
for mu in range(4):
    leftD_dst2 = left_covdev_dst2_from_prop(U_f, seq_bw_prop, mu)
    for nu in range(4):
        gamma_fw = contract("ab, wtzyxbdi j -> wtzyxadi j",
                             D_gammas[nu], prop_fw.data)
        scalar = -0.5 * contract("wtzyxabij, wtzyxbcji, ca -> wtzyx",
                                leftD_dst2, gamma_fw, src_gamma)

# Symmetrize
for mu in range(4):
```

```

for nu in range(mu+1, 4):
    C3_Tmunu[:, mu, nu] = 0.5 * (C3_Tmunu[:, mu, nu]
                                   + C3_Tmunu[:, nu, mu])
    C3_Tmunu[:, nu, mu] = C3_Tmunu[:, mu, nu]

```

The Dirac gamma matrices used in the contraction are $\gamma_1, \gamma_2, \gamma_3, \gamma_4$, indexed by `D_GAMMA_IDS` = [1, 2, 4, 8]. The index `mu=0` corresponds to γ_1 (x -direction in PyQUDA convention), `mu=1` to γ_2 (y), `mu=2` to γ_3 (z), and `mu=3` to γ_4 (time).

Remark 1 (Trace ordering) *The einsum pattern `abij, bcji, ca` should be read as:*

- *`a`: spin-sink of `dst2` (or `leftD_dst2`)*
- *`b`: spin-source of `dst2`, spin-sink of forward propagator*
- *`c`: spin-source of forward propagator, spin index of `src_gamma`*
- *`i, j`: color-sink and color-source, traced by Einstein summation*
- *`ca`: `src_gamma` with indices (spin-source, spin-source-of-gamma)*

The color trace is implicit in the repeated `i` and `j` indices.

3.5 Fixed-Sink Sequential Source Construction

The fixed-sink method absorbs the sum over the sink position y into a sequential propagator. This avoids an $O(V^2)$ summation and makes the three-point calculation tractable.

Step 1: Build the source-smeared, sink-smeared forward propagator.

Starting from the source-smeared point-sink forward propagator `prop_fw_SP(y, x0)`, optional sink smearing is applied:

$$\text{prop_fw_SS}(y, x_0) = \text{boosted_smearing}(\text{prop_fw_SP}(\cdot, x_0), \text{boost} = \text{pos_boost})(y). \quad (14)$$

Step 2: Restrict to the sink time slice and apply momentum phase.

The function `create_meson_bw_seq_pyquda` (in `bw_seq_pyquda.py`) constructs the sequential source:

$$\eta_{\text{seq}}(y; p_f, t_{\text{sep}}) = \delta_{t_y, t_0 + t_{\text{sep}}} \Phi_{p_f}(y - x_0) \Gamma_{\text{seq}} \text{prop_fw_SS}(y, x_0), \quad (15)$$

where the time-slice restriction is implemented via `pyquda.utils.source.sequential12`, and

$$\Gamma_{\text{seq}} = \gamma_5 \Gamma_{\text{sink}}^\dagger \gamma_5. \quad (16)$$

In the einsum implementation:

```

gamma_seq = einsum("ab, cb, cd -> ad", G5, sink_gamma.conj(), G5)
seq_data = einsum("wtzyx, wtzyxilab -> wtzyxilab", mom_phase, seq_data)
seq_data = einsum("ij, wtzyxjlab -> wtzyxilab", gamma_seq, seq_data)

```

Step 3: Invert the Dirac operator.

$$D S_{\text{seq}} = \eta_{\text{seq}}. \quad (17)$$

Step 4: Form the backward sequential meson line.

Using `gamma5` hermiticity, the line that connects the insertion to the sink is

$$S_{\text{seq_anti}}(x; p_f, t_{\text{sep}}) = \gamma_5 S_{\text{seq}}(x)^\dagger \gamma_5. \quad (18)$$

This is computed in `_make_dst2`:

```
dst2 = contract("ab, wtzyxbcij, cd -> wtzyxadij",
               G5, prop.data.conj().transpose(0,1,2,3,4,6,5,8,7), G5)
```

The `transpose(0,1,2,3,4,6,5,8,7)` swaps spin-source/sink (indices 5 $\leftrightarrow$ 6) and color-source/sink (indices 7 $\leftrightarrow$ 8), implementing the Hermitian conjugate with the correct index layout.

Step 5: Post-sequential three-point function.

After the sequential inversion, the sink-summed three-point function takes the compact form:

$$C_3^g(q, \tau; p_f, t_{\text{sep}}) = \sum_{\vec{x}} \Phi_q(x - x_0) \text{Tr}_{sc} \left[S_{\text{seq_anti}}(x; p_f, t_{\text{sep}}) \mathcal{O}_g(x) S_q(x, x_0) \Gamma_{\text{src}} \right]. \quad (19)$$

This is the master formula used by both `get_C3_chi` and `get_C3_Tmunu_symmetrized`.

3.6 Symmetric Covariant Derivative on a Propagator

The symmetric (improved) covariant derivative acting on a quark propagator is defined on the lattice as

$$D_{\mu}^{\text{sym}} S(x) = \frac{1}{2} \left[U_{\mu}(x) S(x + \hat{\mu}) - U_{-\mu}(x) S(x - \hat{\mu}) \right], \quad (20)$$

where $U_{-\mu}(x) = U_{\mu}^{\dagger}(x - \hat{\mu})$. The forward direction $+\mu$ corresponds to direction index $\mu \in \{0, 1, 2, 3\}$, and the backward direction $-\mu$ corresponds to $\mu + 4 \in \{4, 5, 6, 7\}$ in the QUDA direction enumeration.

The code (`_covdev_sym_prop`) applies this spin-color component by component via `MultiFermion`:

```
mf = propagatorToMultiFermion(prop)
for spin in range(4):
    for color in range(3):
        idx = spin * 3 + color
        Dp = U.pure_gauge.covDev(mf[idx], mu)      # +mu
        Dm = U.pure_gauge.covDev(mf[idx], mu + 4)  # -mu
        mf_covdev[idx] = 0.5 * (Dp - Dm)
```

Remark 2 (Sign convention) *The minus sign between D_+ and D_- in the code, $0.5 \times (D_+ - D_-)$, corresponds to the standard lattice symmetric derivative where `covDev` implements the forward/backward parallel transport without the link division by $2a$. The factor $1/2$ in the formula provides the continuum normalization.*

3.7 Left Derivative on the Sequential Line

The second term of the quark EMT insertion requires the left-acting derivative on the backward sequential line $S_{\text{seq_anti}}$. Rather than implementing a separate left-derivative routine, the code exploits gamma5 hermiticity:

$$\overleftarrow{D}_{\mu} S_{\text{seq_anti}}(x) = \gamma_5 (D_{\mu} S_{\text{seq}}(x))^{\dagger} \gamma_5. \quad (21)$$

The derivation is:

$$\overleftarrow{D}_{\mu} S_{\text{seq_anti}} = \overleftarrow{D}_{\mu} (\gamma_5 S_{\text{seq}}^{\dagger} \gamma_5) \quad (22)$$

$$= \gamma_5 (D_{\mu} S_{\text{seq}})^{\dagger} \gamma_5, \quad (23)$$

where the derivative passes through the constant gamma matrices and acts on $S_{\text{seq}}^{\dagger}$ from the right, which is equivalent to the Hermitian conjugate of the right-acting derivative.

The implementation in `_left_covdev_dst2_from_prop` is:


```

D_y = covdev_sym_prop(U_f, prop, mu) # D_mu S_seq
D_y_dag = D_y.data.conj().transpose(0,1,2,3,4,6,5,8,7) # (D_mu S_seq)^dag
leftD_dst2 = contract("ab, wtzyxbcij, cd -> wtzyxadij",
                      G5, D_y_dag, G5)

```

The resulting `leftD_dst2` encodes $\overleftarrow{D}_\mu S_{\text{seq-anti}}(x)$ and is used directly in the second-term contraction of Eq. (11).

3.8 Gradient Flow

Gradient flow is applied to smooth UV fluctuations. The flow radius is approximately $\sqrt{8t}$ where t is the flow time. The code supports both Wilson flow and Symanzik (Lüscher-Weisz) flow via the `flow_type` parameter.

3.8.1 Flow schedule

The flow schedule follows a **measure-before-flow** convention:

1. **Step 0:** Measure observables on the unflowed fields. Output index `step=0` is unflowed.
 2. **First interval (step 0 $\rightarrow$ 1):** Advance fields by 10 small sub-steps of size $\varepsilon/10$.
 3. **Subsequent intervals:** Advance by one step of size ε .
- Thus output index `step` corresponds to flow time $t = \text{step} \times \varepsilon$.

3.8.2 Flowing propagators together

The forward propagator and sequential backward propagator are flowed simultaneously as a single `MultiField` on the flowed gauge background:

```

mf_a = propagatorToMultiFermion(prop_a)
mf_b = propagatorToMultiFermion(prop_b)
packed = multiField(mf_a_all + mf_b_all)
packed_flow = U_f.gradientFlow(packed, flow_type, Nsteps, stepsize)

```

This ensures both fermion fields see the same flowed gauge links at every step.

3.8.3 Gauge field flow

The gauge field itself is flowed by `U_f.wilsonFlow()` or `U_f.symanzikFlow()`, which modifies the gauge links in place. Before flowing, anti-periodic boundary conditions in time are set via `U_f.setAntiPeriodicT()`.

3.9 Connected 3pt High-Level Algorithm

The `connected_3pt` method orchestrates the full measurement:

1. Build source-smeared point-sink forward and backward propagators (`_make_meson_source_props`).
2. Contract the meson 2pt function as a diagnostic (`contract_meson_2pt`).
3. For each source-sink separation t_{sep} :
 - (a) Build and invert the meson fixed-sink sequential source (`create_meson_bw_seq_pyquda`).
 - (b) For each flow step:
 - i. Compute $C_3^X(q, \tau)$ via `get_C3_chi`.
 - ii. Compute $C_3^{\mu\nu, q}(q, \tau)$ via `get_C3_Tmunu_symmetrized`.
 - iii. Advance the flowed propagators via `_advance_floweds_props`.
4. Save the results to HDF5.

3.10 HDF5 Output Format (Connected 3pt)

The output file (written by `save_empt_quark_3pt_hdf5`) contains:

- `C2`: 1D array of length N_t – two-point function at the selected sink gamma and zero momentum, for normalization.
- `C3_chi`: 4D array $[N_{ts}, N_{steps}+1, N_q, N_t]$ – scalar diagnostic.
- `C3_Tmunu`: 6D array $[N_{ts}, N_{steps}+1, N_q, 4, 4, N_t]$ – quark EMT 3pt.
- `momentum_transfer_list`: list of $[q_x, q_y, q_z, q_t]$ arrays.
- HDF5 attributes: `measurement`, `spin`, `flow_type`, `flow_epsilon`, `flow_steps`, `n_t_separations`, `src_interpolator`, `sink_interpolator`, `contraction_convention` (“B”), `meson_sign` (+1), `n_qext`.

The meson 2pt diagnostic is saved separately by `save_empt_meson_2pt_hdf5` with:

- `C2`: 3D array $[16, N_{mom}, N_t]$ – 16 sink gammas $\times$ all momenta $\times$ time.
- `gamma_list`: list of gamma labels.
- `momentum_list`: list of momentum vectors.

4 Part II: Disconnected One-Point Functions

4.1 Stochastic Quark One-Point Function

The disconnected quark loop requires the trace of the quark propagator with operator insertions. These are estimated stochastically using $\mathbb{Z}_n$ noise vectors.

4.1.1 Noise generation

A $\mathbb{Z}_n$ noise fermion $\xi(x)$ is generated with random complex phases:

$$\xi(x) \in \{e^{2\pi i r/n} \mid r \in \{0, 1, \dots, n-1\}\}, \quad (24)$$

with $n = n_{zn}$ (typically $n = 2$, i.e., $\mathbb{Z}_2$ noise). Each spin-color component at each lattice site receives an independent random phase. The noise satisfies $\langle \xi(x) \rangle = 0$ and $\langle \xi(x) \xi^\dagger(y) \rangle = \delta_{xy}$.

The solution vector is obtained by inverting the Dirac operator:

$$\eta(x) = D^{-1} \xi(x). \quad (25)$$

4.1.2 CHI diagnostics

Two scalar diagnostics are computed to monitor the stochastic estimator:

$$\text{CHI}[0](q, t) = \sum_{\vec{x}} \Phi_q(x) \xi^\dagger(x) \eta(x) \quad (26)$$

$$\text{CHI}[1](q, t) = \sum_{\vec{x}} \Phi_q(x) \xi^\dagger(x) \xi(x) \quad (27)$$

$\text{CHI}[0]$ is the stochastic estimate of $\text{Tr}(D^{-1})$ and should average to the scalar vacuum condensate. $\text{CHI}[1]$ is the noise normalization (for $\mathbb{Z}_2$ noise, $\xi^\dagger \xi = \text{constant}$ at each site).

In code:

```
dot_xi_eta = contract("etzyxbc, etzyxbc -> etzyx",
                      xi.data.conj(), eta.data)
CHI[0] = impose_P_Breit_slice(U_f, dot_xi_eta, phases_3pt)

dot_xi_xi = contract("etzyxbc, etzyxbc -> etzyx",
                     xi.data.conj(), xi.data)
CHI[1] = impose_P_Breit_slice(U_f, dot_xi_xi, phases_3pt)
```

4.1.3 Quark EMT one-point building block

The stochastic estimate of the quark EMT one-point function is

$$T_{\nu\mu}^q(q, t) = -\frac{1}{2} \sum_{\vec{x}} \Phi_q(x) \xi^\dagger(x) \gamma_\nu [D_{+\mu} - D_{-\mu}] \eta(x). \quad (28)$$

Here $D_{\pm\mu}$ are the forward/backward covariant derivatives (NOT the symmetrized versions – the difference $D_+ - D_-$ already implements the symmetric derivative up to a factor of 2). The indices on $T_{\nu\mu}^q$ are ordered (ν, μ) because the gamma matrix index comes first in the code:

```
for mu in range(4):
    tmp = covDev(eta, mu) - covDev(eta, mu + 4)    # (D_+mu - D_-mu) eta
    for nu in range(4):
        Y = contract("ab, ...bc -> ...ac", D_gammas[nu], tmp.data)
        complex_field = contract("...sc, ...sc -> ...",
                                   xi.data.conj(), Y)
        Tmunu[nu, mu] += -0.5 * impose_P_Breit_slice(...)
```

The minus sign $(-1/2)$ follows from Eq. (9) combined with the stochastic trace convention.

4.1.4 Symmetrization, noise averaging, and volume normalization

After accumulating all μ, ν components, the tensor is symmetrized (identical to the connected case):

$$T_{\mu\nu}^q \rightarrow \frac{1}{2} (T_{\mu\nu}^q + T_{\nu\mu}^q). \quad (29)$$

The measurements are repeated for `n_vec` independent noise vectors and averaged:

$$\langle T_{\mu\nu}^q \rangle = \frac{1}{N_{\text{vec}}} \sum_{v=1}^{N_{\text{vec}}} T_{\mu\nu}^{q,(v)}. \quad (30)$$

Finally, the result is divided by the spatial volume $V_3 = L_x L_y L_z$ to obtain the intensive one-point density:

$$T_{\mu\nu}^{q,\text{avg}}(q, t, t_f) = \frac{1}{V_3} \langle T_{\mu\nu}^q(q, t, t_f) \rangle. \quad (31)$$

4.1.5 HDF5 output format (Quark 1pt)

The file (written by `save_emt_quark_1pt_hdf5`) contains:

- `raw/Tmunu_pervec`: 6D [`n_vec`, 4, 4, `Nq`, `Nsteps+1`, `Nt`] – per-noise-vector EMT.
- `raw/CHI_pervec`: 5D [`n_vec`, 2, `Nq`, `Nsteps+1`, `Nt`] – per-noise-vector CHI.
- `avg/Tmunu/T11, T12, ..., T44`: averaged, volume-normalized tensor components (symmetric storage: only $\mu \leq \nu$).
- `avg/CHI`: 4D [2, `Nq`, `Nsteps+1`, `Nt`] – averaged CHI.
- Attributes: `measurement`, `flow_type`, `flow_epsilon`, `flow_steps`, `n_vec`, `n_zn`.

4.2 Ringed Fermion Normalization

The ringed fermion renormalization scheme [3] requires the flowed fermion kinetic expectation value. At zero momentum ($q = 0$), the diagonal sum of the quark EMT one-point provides precisely this quantity:

$$\sum_{\mu=1}^4 T_{\mu\mu}^q(0, t) = -\frac{1}{2} \sum_{\vec{x}} \xi^\dagger(x) \gamma_\mu [D_{+\mu} - D_{-\mu}] \eta(x) \quad (32)$$

$$\propto \langle \bar{\chi}(t) \overleftrightarrow{D} \chi(t) \rangle, \quad (33)$$

up to lattice discretization effects and the stochastic estimator conventions. In the saved quark 1pt HDF5 file, this quantity can be reconstructed from

$$\text{avg/Tmunu/T11, T22, T33, T44}$$

at the $q = 0$ momentum index. Note that the CHI datasets are the scalar trace and noise diagnostics, *not* the ringed fermion normalization itself.

4.3 Gluon One-Point Function

4.3.1 Clover field strength

The gauge field strength $F_{\mu\nu}(x)$ is constructed via the clover (plaquette + rectangle) operator. The standard clover loop for the (μ, ν) plane combines four oriented paths:

$$\begin{aligned} \text{loop}_1 &: \mu \rightarrow \nu \rightarrow -\mu \rightarrow -\nu, \\ \text{loop}_2 &: \nu \rightarrow -\mu \rightarrow -\nu \rightarrow \mu, \\ \text{loop}_3 &: -\mu \rightarrow -\nu \rightarrow \mu \rightarrow \nu, \\ \text{loop}_4 &: -\nu \rightarrow \mu \rightarrow \nu \rightarrow -\mu. \end{aligned} \quad (34)$$

The field strength is extracted as

$$F_{\mu\nu}(x) = \frac{1}{8} (Q_{\mu\nu}(x) - Q_{\mu\nu}^\dagger(x)), \quad (35)$$

where $Q_{\mu\nu}$ is the sum of the four clover loops (each with unit coefficient). The resulting $F_{\mu\nu}$ is anti-Hermitian by construction.

4.3.2 Traceless projection

The field strength is projected onto the traceless anti-Hermitian $\mathfrak{su}(3)$ algebra:

$$F_{\mu\nu}^{\text{su}(3)}(x) = F_{\mu\nu}(x) - \frac{1}{N_c} \text{Tr}_c [F_{\mu\nu}(x)] \cdot \mathbf{1}_{N_c}. \quad (36)$$

The code then multiplies by $-i$ to match the Euclidean convention:

$$F_{\mu\nu}^{\text{Eucl}}(x) = -i F_{\mu\nu}^{\text{su}(3)}(x). \quad (37)$$

In `_F_clover_traceless`:

```
A = 0.125 * (data - data.swapaxes(-2, -1).conjugate())
trA = contract("...ii -> ...", A)
A -= trA[...], None, None] * I / Nc
data[...] = (-1j) * A
```

4.3.3 Gluon EMT building block

The gluonic EMT one-point function is

$$T_{\mu\nu}^g(q, t) = \frac{2}{V_3} \sum_{\vec{x}} \Phi_q(x) \sum_{\substack{\rho=0 \\ \rho \neq \mu, \nu}}^3 \text{Tr}_c \left[F_{\mu\rho}(x) F_{\nu\rho}(x) \right]. \quad (38)$$

The sum over ρ runs over all Lorentz indices except μ and ν . This is the full gluonic EMT building block (not a traceless projection of the EMT itself – the traceless projection in `_F_clover_traceless` operates only on the field-strength level to ensure $\mathfrak{su}(3)$ algebra).

In code:

```
for mu in range(4):
    for nu in range(mu, 4):
        tmp = zeros(...)
        for rho in range(4):
            if rho == mu or rho == nu: continue
            tmp += contract("...ab, ...ba -> ...", F[mu][rho], F[nu][rho])
        Tmunu_t[mu, nu, :, step, :] += 2.0 * slice_t
```

The factor 2 is explicit in the formula, and the volume normalization $1/V_3$ occurs after the spatial Fourier sum.

Antisymmetry: $F_{\mu\nu}$ is antisymmetric by construction, so only the six independent planes $(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)$ are computed, and $F_{\nu\mu} = -F_{\mu\nu}$ is filled by antisymmetry.

4.3.4 HDF5 output format (Gluon 1pt)

The file (written by `save_empt_gluon_1pt.hdf5`) contains:

- `Tmunu/T11, T12, ..., T44`: 3D arrays `[Nq, Nsteps+1, Nt]` – volume-normalized tensor components (symmetric storage: only $\mu \leq \nu$).
- Attributes: `measurement, flow_type, flow_epsilon, flow_steps`.

4.4 Gradient Flow Schedule (One-Point)

The one-point flow schedule mirrors the connected 3pt schedule (measure-before-flow, step-0 is unflowed). There is one important difference for the quark 1pt: the noise vector ξ and solution η are flowed together as a single `MultiField`. This ensures that the flowed ξ and η correspond to the same flowed gauge background.

Quark 1pt flow:

```
for step in range(Nsteps + 1):
    measure Tmunu, CHI
    if Nsteps > 0 and step == 0:
        temp = multiField([xi, eta])
        temp_flow = U_f.gradientFlow(temp, flow_type, 10, stepsize/10)
        xi, eta = temp_flow[0], temp_flow[1]
    elif Nsteps > 0 and step < Nsteps:
        temp = multiField([xi, eta])
        temp_flow = U_f.gradientFlow(temp, flow_type, 1, stepsize)
        xi, eta = temp_flow[0], temp_flow[1]
```

Gluon 1pt flow:

```

for step in range(Nsteps + 1):
    measure Tmunu
    if Nsteps > 0 and step == 0:
        U_flow.wilsonFlow(10, epsilon=stepsize/10) # or symanzik
    elif Nsteps > 0 and step < Nsteps:
        U_flow.wilsonFlow(1, epsilon=stepsize)      # or symanzik

```

4.5 Renormalization Assembly (Analysis Stage)

The contraction kernel outputs *bare flowed operators*. The final renormalized EMT is assembled in the analysis stage:

$$T_{\mu\nu}^{\text{ren}}(x) = c_g(t) O_{\mu\nu}^g(t, x) + c_q(t) O_{\mu\nu}^q(t, x) + c_s(t) [O_{\mu\nu}^q(t, x)]_{\text{traceless}} + \delta_{\mu\nu} c_{\text{trace}}(t) \text{Tr} [O(t, x)] + \dots \quad (39)$$

where:

- $O_{\mu\nu}^g(t, x)$ is from the gluon 1pt measurement (Sec. 4.3).
- $O_{\mu\nu}^q(t, x)$ is from the quark 1pt measurement (Sec. 4.1) for the disconnected piece, and from the connected 3pt measurement (Sec. 3.4) for the connected piece.
- $c_g(t)$, $c_q(t)$ are perturbative matching coefficients computed at small flow time.
- Traceless projections and trace anomaly subtraction terms are handled in analysis.
- Vacuum subtraction (for the disconnected diagrams) is performed per gauge configuration.

The disconnected matrix element is formed in analysis as

$$C_3^{\text{disc}} = \langle C_2 L \rangle - \langle C_2 \rangle \langle L \rangle, \quad (40)$$

where L is the quark or gluon loop (1pt) and C_2 is the meson two-point function. The quark 1pt and gluon 1pt files provide L , while the meson 2pt file provides C_2 .

4.6 Future Variance Reduction Strategies

The current implementation uses plain $\mathbb{Z}_n$ noise for the stochastic quark 1pt estimator. Two natural upgrades are identified for future production:

1. **Hierarchical probing** [4] (arXiv:1302.4018): The EMT 1pt trace estimator is $\text{Tr}(\Gamma D^{-1})$ with local gamma and derivative insertions. Hierarchical probing replaces or supplements random noise vectors with distance-ordered Hadamard/coloring vectors on the 4D torus. This cancels near-diagonal contributions of D^{-1} more deterministically, reducing stochastic variance at fixed inversion count. The method is particularly effective for operators with local support like $T_{\mu\nu}$.
2. **Frequency splitting / propagator decomposition** (arXiv:2605.00643): The flowed EMT loop can be decomposed into low/infrared and high/ultraviolet components using frequency-filtered estimator pieces. This allows expensive noise averaging to be focused on the component with the largest residual variance. The strategy should be designed together with the gradient-flow radius since flow already smooths ultraviolet modes.

When implemented, these upgrades should preserve the existing HDF5 schema or add explicit estimator metadata so that disconnected diagram analysis can combine old and new 1pt data safely.

4.7 Summary of All HDF5 Output Files

Notation: Np = number of momenta, Nq = number of momentum transfers, Nts = number of t_{sep} values, Ns = number of flow steps, Nv = number of noise vectors, Nt = temporal lattice extent.

Table 1: Summary of HDF5 output files produced by the pion EMT workflow.

File type	Tag subdirectory	Main datasets	Shape
Meson 2pt	EMT2pt/	C2	[16, Np, Nt]
Quark 3pt	EMT3pt/	C2	[Nt]
		C3_chi	[Nts, Ns+1, Nq, Nt]
		C3_Tmunu	[Nts, Ns+1, Nq, 4, 4, Nt]
Quark 1pt	EMTc/	raw/Tmunu_pervec	[Nv, 4, 4, Nq, Ns+1, Nt]
		avg/Tmunu/Tij	[Nq, Ns+1, Nt]
Gluon 1pt	EMTg/	Tmunu/Tij	[Nq, Ns+1, Nt]

5 Technical Implementation Details

5.1 Parameter Dictionary

The QuarkEMT and GluonEMT classes are initialized with a parameter dictionary containing:

Key	Description
qext	list of momentum transfers $[q_x, q_y, q_z, q_t]$
pf	fixed sink momentum $[p_x, p_y, p_z, p_t]$
p_2pt	list of momenta for the 2pt function
CG_GaussSmear	whether to apply Gaussian smearing
pos_boost	momentum boost for forward source/sink
neg_boost	momentum boost for backward source/sink
width	Gaussian smearing width
flow_type	"wilson" or "symanzik"
flow_epsilon	flow step size ϵ
flow_steps	number of flow steps

5.2 Inverter Parameters

The Dirac operator uses the QUDA multigrid solver with parameters:

```
invPara = [mass, csw, tol, maxiter]
```

Typical values: `mass = 0.236`, `csw = 1.0372`, `tol = 1e-15`, `maxiter = 300`. The multigrid setup uses a `[[8, 8, 4, 4]]` block size configuration.

5.3 Random Number Parameters (Quark 1pt)

`randPara = [n_vec, n_zn, randseed]` where `n_vec` is the number of noise vectors, `n_zn` is the $\mathbb{Z}_n$ order (typically 2), and `randseed` is used to seed the random number generator (usually set to the configuration number for reproducibility).

5.4 Typical Usage Pattern

The application scripts follow a common pattern:

1. **Initialize QUDA:** `init(mpi_geometry, enable_mps=True)`
2. **Read gauge field:** `io.readNERSCGauge(gauge_path)`
3. **Apply HYP smearing:** `gauge.hypSmear(1, 0.75, 0.6, 0.3, 4)`
4. **Set boundary condition:** `gauge.latt_info.t.boundary = -1` (anti-periodic in time)
5. **Instantiate measurement class:** `quark_empt = QuarkEMT(parameters)`

6. **Run measurement:** `quark_empt.connected_3pt(...)`
7. **Results saved to HDF5** automatically.

6 Cross-Checks and Consistency

- **Convention B + meson_sign = +1:** All meson 2pt and 3pt contractions are internally consistent with this choice. Zero-momentum regression tests match previous baselines to roundoff precision.
- **CHI diagnostic:** C_3^χ with $\mathcal{O}_g = \mathbf{1}$ must be numerically consistent with the meson 2pt function (up to a sink momentum phase factor). This is a strong check of the sequential source construction.
- **Symmetrization:** Both connected and disconnected EMT tensors are symmetrized $\mu \leftrightarrow \nu$ after the full contraction. The antisymmetric part should be zero within statistics.
- **Flow schedule alignment:** The quark and gluon flow schedules are aligned (measure-before-flow, first interval subdivided). Output index **step** corresponds to flow time $t = \text{step} \times \varepsilon$ in all three file types.
- **Volume normalization:** The connected 3pt output is NOT volume-normalized (it is an extensive correlator). The 1pt outputs ARE divided by V_3 (intensive densities). This distinction is critical for the analysis stage when forming $\langle C_2 L \rangle$.

References

- [1] M. Lüscher, “Properties and uses of the Wilson flow in lattice QCD,” JHEP **08** (2010), 071. [arXiv:1006.4518 [hep-lat]].
- [2] M. Lüscher and P. Weisz, “Perturbative analysis of the gradient flow in non-abelian gauge theories,” JHEP **02** (2011), 051. [arXiv:1101.0963 [hep-th]].
- [3] H. Makino and H. Suzuki, “Lattice energy-momentum tensor from the Yang-Mills gradient flow – inclusion of fermion fields,” PTEP **2014** (2014), 063B02. [arXiv:1404.2758 [hep-lat]].
- [4] A. Stathopoulos, J. Laeuchli, and K. Orginos, “Hierarchical probing for estimating the trace of the matrix inverse on toroidal lattices,” [arXiv:1302.4018 [hep-lat]].
- [5] L. Del Debbio, A. Patella, and C. Pica, “Energy-momentum tensor on the lattice: recent developments,” [arXiv:2105.08278 [hep-lat]].
- [6] T. Harris, G. von Hippel, P. Junnarkar, H. B. Meyer, K. Ottnad, J. Wilhelm, H. Wittig, and L. Wrang, “Nucleon isovector tensor form factors from the lattice,” Phys. Rev. D **100** (2019), 034513. [arXiv:1905.01291 [hep-lat]].
- [7] H. Suzuki, “Energy-momentum tensor from the Yang-Mills gradient flow,” PTEP **2013** (2013), 083B03. [arXiv:1304.0533 [hep-lat]].